

Centre for Development of Advanced Computing (C-DAC), Mumbai
PGCP-AC – August 26 Batch
Lab Examination: Object-Oriented Programming with Java (OOPJ)

Duration: 120 Minutes

Question 1 — Student Result Management System

Create a console-based Java program to manage and analyze student examination results. Each Result record should include:

Class Result

- Student ID (unique, auto-generated)
- Student Name
- Subject Name
- Marks Obtained (int, out of 100)
- Grade (use Enum: DISTINCTION, FIRST_CLASS, SECOND_CLASS, PASS, FAIL)
- Exam Date (use LocalDate)

Program should provide the following functionalities:

1. **Add a new Result entry:** Allow the user to add a student result. Student ID should be auto-generated. Grade must be assigned automatically based on marks (≥ 75 : DISTINCTION, ≥ 60 : FIRST_CLASS, ≥ 50 : SECOND_CLASS, ≥ 40 : PASS, else FAIL). Store in memory.
2. **Display results for a specific Grade:** Use a Predicate<Result> (named byGrade) to filter results by a given Grade Enum value and display using Stream API.
3. **Calculate average marks by Subject:** Use a Predicate<Result> (named bySubject) to filter results for a given subject and compute the average marks using Stream API.
4. **Find the Top Scorer:** Use the Stream API with Optional<Result> to find and display the student with the highest marks. If no results exist, display an appropriate message using Optional.orElse().
5. **Display all results sorted by Marks (Descending):** Use Stream API with Comparator to sort and display all results in descending order of marks obtained.

Note:

- Use appropriate Java collections like List<Result> to store result entries in memory.

- Use Enum Grade and assign grade automatically during result creation.
- Use Predicate<Result> to encapsulate reusable filter conditions (byGrade, bySubject).
- Use Java 8 Predicate, Stream API, Optional, Comparator, and Lambda expressions where applicable.
- Follow Java conventions when naming packages, classes, methods, variables and modularity.
- Implement custom exception handling for invalid inputs such as marks out of range or empty fields.

Question 2 — Bank Account System with Custom Exception

Banks maintain different types of accounts such as **Savings Accounts** and **Current Accounts**. Each account allows withdrawals, but the system must ensure that the account balance never becomes negative.

If a withdrawal amount exceeds the available balance, the system should throw a **custom exception**.

Create the following classes:

1. Account (Base Class)

Attributes:

- accountNumber (int)
- balance (double)

Methods:

- withdraw(double amount)

The method should attempt to withdraw money from the account.

2. SavingsAccount (Derived Class)

Inherits from **Account**.

Rule:

- Savings account must maintain a minimum balance of 500.
- If withdrawal causes the balance to go below 500, throw an exception.

3. CurrentAccount (Derived Class)

Inherits from **Account**.

Rule:

- Balance should not become negative.
- If withdrawal exceeds balance, throw an exception.

Custom Exception

Create a custom exception:

- InsufficientBalanceException

Message:

“Withdrawal denied due to insufficient balance”

Task

1. Read account type and details.
2. Create an account object using **polymorphism**.
3. Perform withdrawal.
4. Handle the custom exception if it occurs.

Input Format

Field	Description
accountType	savings / current
accountNumber	integer
balance	decimal
withdrawAmount	decimal

Output Format

If withdrawal successful:

Withdrawal successful

Remaining Balance = <balance>

If exception occurs:

Withdrawal denied due to insufficient balance

Sample Test Cases

Case	Details
Sample 1	Input: savings, 101, 2000, 1000 Output: Withdrawal successful Remaining Balance = 1000

Case	Details
Sample 2	Input: savings, 101, 1000, 600 Output: Withdrawal denied due to insufficient balance (Explanation: Savings account must maintain minimum balance of 500.)
Sample 3	Input: current, 102, 1000, 1200 Output: Withdrawal denied due to insufficient balance

Constraints

- $1 \leq \text{accountNumber} \leq 100000$
- $0 \leq \text{balance} \leq 100000$
- $0 \leq \text{withdrawAmount} \leq 100000$